Exercise 12

Elisa Schöpf, Leon Krieger, Nikola Adzic

June 14, 2026

Contents

1 Setup and Basic Execution **1**

2 Profiling **1**

3 Code Understanding **2**

4 Optimization **3**
4.1 Compiler flags 3
4.2 Interpreter fast-path for `OP_CALL` 3

1 Setup and Basic Execution

LCC3 fib.lua results:

```
100 x fibonacci_naive(30)     time:  13.7866 s  --  832040
10000000 x fibonacci_tail(30) time:  15.8393 s  --  832040
25000000 x fibonacci_iter(30) time:  12.2847 s  --  832040
```

The median of averages from 20 runs was 13.7609 s

2 Profiling

Initially we used `profile.lua`(`https://github.com/2dengine/profile.lua`) to profile the benchmark. However this only profiles the Lua script itself, showing which Lua functions take the most time. It confirmed that the Fibonacci functions are the main workload. Since we are not allowed to modify the Lua script, knowing which script-level function is slow does not give us any starting point for optimizing the interpreter.

We then used `perf` to profile the program. This result was more useful because it breaks down the time spent in the C source code.

```
[...]
# Samples: 116K of event 'cycles:u'
# Event count (approx.): 114068069768
#
# Children      Self  Command  Shared Object     Symbol
# ........  ........  .......  .................  ......................
#
    79.12%    78.18%  lua      lua               [.] luaV_execute
            |
            |--14.88%--0
            |          |
            |           --14.61%--luaV_execute
            |
            |--7.37%--0x281c900060080
            |          luaV_execute
[...]
```

The programs spends ∼79% of its time in `luaV_execute`, which is the core VM loop, meaning we should focus our efforts there. It also shows some time is spent in `luaD_precall` and `luaD_pretailcall` (∼15% combined). This tells us that the overhead isn't just in executing instructions, but also in the management of the Lua stack and function calls.

We also manually added opcode statistics to get a closer look at what `luaV_execute` does internally, which tells us what the hottest parts of the code are and gives us hints where our optimizations could start.

```
--- Opcode Statistics ---
EQI             : 310000031
LTI             : 271946237
GETTABUP        : 135973068
ADD             : 135973068
MOVE            : 35000103
LOADI           : 25000001
CLOSURE         : 10000001
VARARGPREP      : 4
SETTABUP        : 3
---------------------------
```

3 Code Understanding

There are two major phases in the Lua interpreter, script compilation and script execution. The interpreter first goes through the program code to convert it into Lua bytecode instructions. This bytecode is then executed by Lua's virtual machine. The majority of it is handled by the `luaV_execute` function. This function consists of a large switch statement which checks against different OP_IDs to determine what to do.

The compile time was measured by using `luac`, which only compiles the program and doesn't execute it. This pre-compiled bytecode was then used to measure the execution time. We can clearly see that the compilation time for the benchmark is minuscule compared to the execution time.

```
Compilation Median:     00.0083 sec
Execution Median:       37.4024 sec
```

As seen on the `perf` report, the interpreter spends a lot of time in `luaV_execute`. As expected, this takes a lot of time and is inefficient.
`LUA_USE_JUMPTABLE` helps with that by enabling direct-threaded dispatch in the virtual machine using GCC/Clang computed gotos. This replaces the large switch statement, thus increasing performance. The jumptable only works with gcc/clang and is turned automatically on when compiled with either of those. It can be turned off by passing `-DLUA_USE_JUMPTABLE=0` to the compiler.

```
 Function                        :  With      :  Without   :  Speedup :
100 x fibonacci_naive(30)        :  13.7866 s :  13.8432 s :  x 1.004 :
10000000 x fibonacci_tail(30)    :  15.8393 s :  19.2325 s :  x 1.214 :
25000000 x fibonacci_iter(30)    :  12.2847 s :  14.7320 s :  x 1.199 :
Median of averages               :  13.7609 s :  14.7412 s :  x 1.071 :
```

The speedup column reveals how much faster the program is executed when gotos are utilized. The performance gains do differ across different implementations. The recursive implementation `fibonacci_naive` shows barley any improvement, most likely due the overhead of the function constantly calling itself and managing those recursive layers. Both the tail-recursive and iterative implementations see a speedup of ∼20%. They spend most of their time executing the same small sequence of operations over and over again, which the jumptable improves.

4 Optimization

Each configuration was run 5 times. For every benchmark variant (`naive`, `tail`, `iter`) we take the median over the runs and report `AVG` as the mean of those three medians. The median guards against single slow runs, and averaging over all three variants stops an optimization that only helps one variant while hurting the others from looking like an overall win.

4.1 Compiler flags

Before touching the interpreter source we tested several compiler configurations to see how much performance build flags alone can buy.

config	naive	tail	iter	AVG	vs base
baseline (-O2)	11.868	15.899	11.374	13.047	--
O3	12.155	16.146	11.410	13.237	+1.5%
O3 -march=native	12.194	16.066	12.049	13.437	+3.0%
O3 -fno-gcse -fno-crossjumping	11.952	14.852	10.462	12.422	-4.8%
O3 + PGO	11.901	13.901	10.216	12.006	-8.0%
O3 + PGO -fno-gcse -fno-crossj.	11.892	15.001	9.894	12.262	-6.0%
O3 + PGO -march=native	10.986	14.715	10.503	12.068	-7.5%

Two results stand out. First, going from `-O2` to `-O3` did not help and even made the interpreter slightly slower, with or without `-march=native`. The VM is one tight dispatch loop and the extra inlining and unrolling that `-O3` adds mostly grow the code rather than speed it up. Second, `-fno-gcse -fno-crossjumping` gave a ∼5% speedup. Normally these two passes merge the repeated ends of the opcode handlers into one shared spot. Turning them off keeps a separate jump for each opcode, which the CPU predicts much better.

The only flag that reliably paid off was profile-guided optimization (PGO) at ∼8%. The two speedups do not add up though. Putting `-fno-gcse -fno-crossjumping` on top of PGO is about 2% slower than PGO on its own. The slowdown is all in the `tail` variant (15.00 s vs. 13.90 s), while `iter` gets a little better. PGO already lays out the hot code better than these flags do, so adding them on top only gets in the way. PGO with `-march=native` was also slightly worse than PGO alone, so native code does not help here either. We therefore use plain `-O3 + PGO` for the optimization below.

4.2 Integer fast-path for `OP_ADD`

The benchmark only ever adds integers, so we gave the `OP_ADD` handler a fast path that adds two integer operands directly and skips the following `MMBIN` metamethod check. On its own this gave a ∼1.4% speedup. The added code is shown below.

```
// Apply heuristic of ints
if (ttisinteger(v1) && ttisinteger(v2)) {
  lua_Integer i1 = ivalue(v1);
  lua_Integer i2 = ivalue(v2);
  setivalue(s2v(RA(i)), intop(+, i1, i2));
  pc++; /* skip following MMBIN */
  vmbreak;
}
```

4.3 Interpreter fast-path for `OP_CALL`

The `perf` profile showed ∼15% of the runtime in `luaD_precall` / `luaD_pretailcall`, the helper functions that set up a new call frame. Most calls in the benchmark are just calls to a normal Lua function, which only needs a short, fixed setup, but the code still reaches it through a separate function call and a type check. We moved that common case straight into the `OP_CALL` handler in `luaV_execute`, so it builds the new call frame and jumps right into the called function. All other calls (C functions and `__call` metamethods) still go through the original `luaD_precall`.

```
if (l_likely(ttisLclosure(s2v(ra)))) {
  /* fast path: call to a plain Lua closure -- inline the common
     LUA_VLCL case of luaD_precall, skipping the out-of-line call. */
```

```
  Proto *p = clLvalue(s2v(ra))->p;
  int narg = cast_int(L->top.p - ra) - 1;
  int nfixparams = p->numparams;
  int fsize = p->maxstacksize;
  savepc(ci);
  checkstackp(L, fsize, ra);            /* may move the stack; fixes 'ra' */
  newci = L->ci->next ? L->ci->next : luaE_extendCI(L);
  L->ci = newci;
  newci->func.p = ra;
  newci->callstatus = cast_uint(nresults + 1);
  newci->top.p = ra + 1 + fsize;
  newci->u.l.savedpc = p->code;
  for (; narg < nfixparams; narg++)
    setnilvalue(s2v(L->top.p++));        /* complete missing arguments */
  ci = newci;
  goto startfunc;
}
```

To separate the patch from PGO, we benchmarked three interpreters on the same node interleaved.

```
config                        naive    tail    iter     AVG
baseline (-O2, vanilla)      12.649  16.129  11.990  13.589
PGO only (O3+PGO)            12.798  14.260  10.370  12.476
patched (OP_CALL + O3+PGO)   10.898  13.990  10.137  11.675

                              naive    tail    iter     AVG
patched vs baseline          -13.8%  -13.3%  -15.4%  -14.1%
patched vs PGO-only          -14.8%   -1.9%   -2.2%   -6.4%
(node n002, median of 5 runs; negative = faster)
```

Together with PGO, the patched interpreter is $\sim$14% faster than the `-O2` baseline. The last row shows the patch on its own, which adds another $\sim$6% on top of PGO. That average is misleading though, because almost all of it comes from `naive` (-15%), the recursive version that spends its time on function calls. The `tail` and `iter` versions, which mostly run loops, move only $\sim$2%, which is within the noise between runs. This is what we expected, since the patch only shortens the call path. It helps code that calls a lot and does little for code that stays in a loop. So the fast-path is a clear win for call-heavy code and makes no real difference elsewhere.